Investigación: - Patrones GOF (Gang of Four):

➢ ¿Cómo se dividen los patrones GOF y que significa cada categoría? Realizar una explicación de cada uno de ellos, con su respectivo diagrama de clases y un ejemplo, destacando cuando es conveniente utilizaros y en que contexto.

Los patrones GoF se dividen en tres categorías distintas:

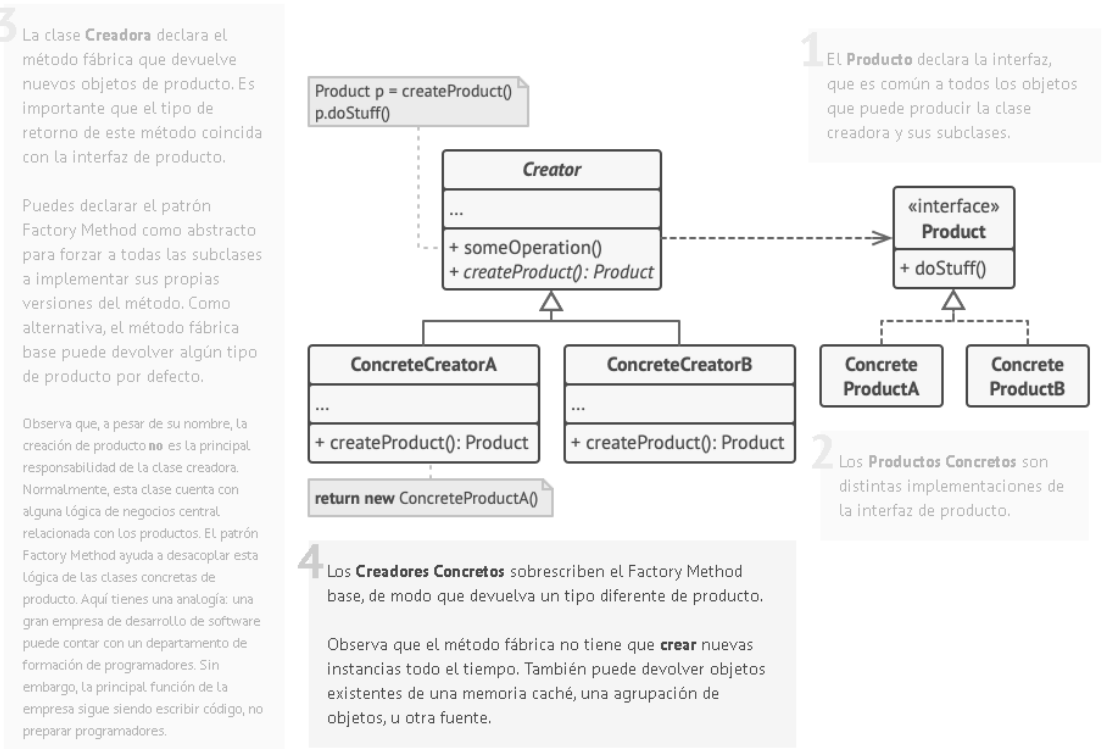
1. **Creacionales:** se centran en mecanismos de creación de objetos

1.1) Factory Method**:** proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.

Resuelve el problema de crear objetos sin especificar la clase exacta del objeto que se va a crear

Cuándo conviene usarlo: cuando no se conoce de antemano el tipo exacto de los objetos con los que debe trabajar el código, o cuando se quiere que sean las subclases quienes decidan qué clase concreta instanciar, de modo que se puedan agregar nuevos productos sin modificar el código existente.

Diagrama:



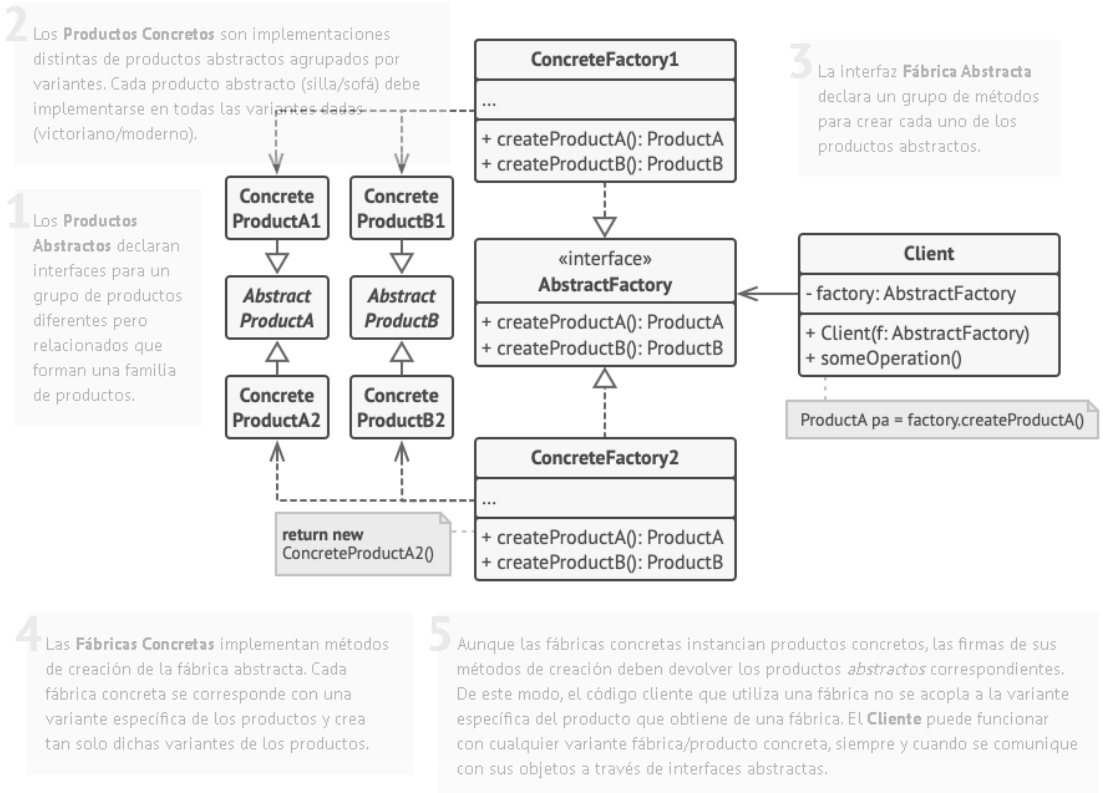
Ejemplo: un sistema de documentos que crea archivos PDF o Word mediante un creador genérico.

1.2) Abstract Factory Method: permite producir familias de objetos relacionados sin especificar sus clases concretas.

A diferencia del *Factory Method* (que se centra en la creación de un solo tipo de objeto mediante subclases), el *Abstract Factory* permite crear familias de objetos relacionados o dependientes entre sí, sin necesidad de especificar sus clases concretas.

El patrón *Abstract Factory* garantiza que todos los productos creados juntos pertenezcan a la misma familia y sean compatibles entre sí.

Cuándo conviene usarlo: cuando el sistema debe trabajar con distintas familias de productos relacionados y es necesario garantizar que se usen juntos los de una misma familia (por ejemplo, un mismo estilo visual o plataforma), sin depender de sus clases concretas.



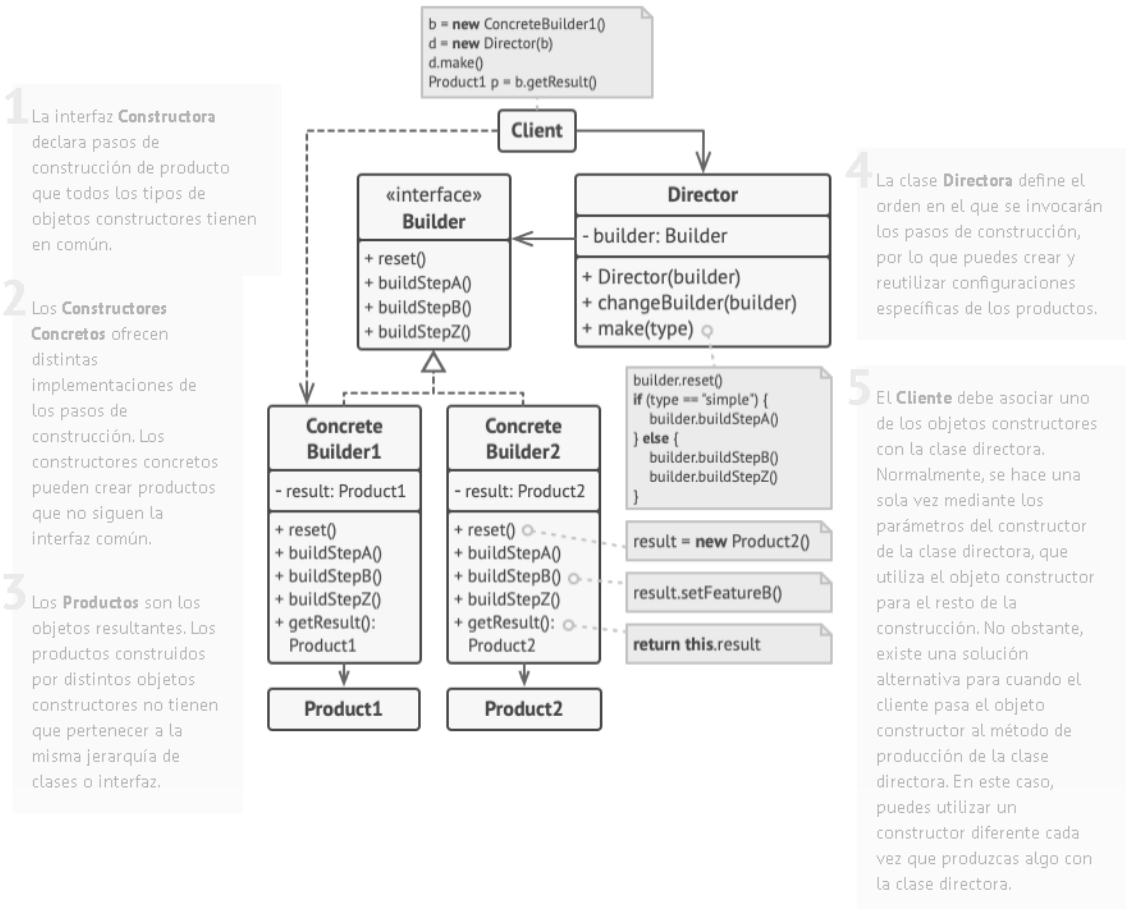
Ejemplo: un kit de interfaz gráfica que genera componentes (botones, menús) estilo Windows o macOS.

1.3) Builder: permite construir objetos complejos paso a paso. El patrón nos permite producir distintos tipos y representaciones de un objeto empleando el mismo código de construcción.

Es bueno aplicarlo en estos casos:

- Cuando un objeto necesita ser creado con una gran cantidad de parámetros de configuración, muchos de los cuales son opcionales o tienen valores por defecto.
- Cuando el proceso de creación del objeto debe permitir diferentes representaciones del mismo tipo.
- Cuando queremos asegurar la inmutabilidad del objeto final (es decir, que una vez creado, sus valores no puedan modificarse).

Diagrama:



Ejemplo: armar una computadora a medida paso a paso (procesador, memoria, disco, placa de video), eligiendo solo los componentes necesarios y dejando valores por defecto para el resto.

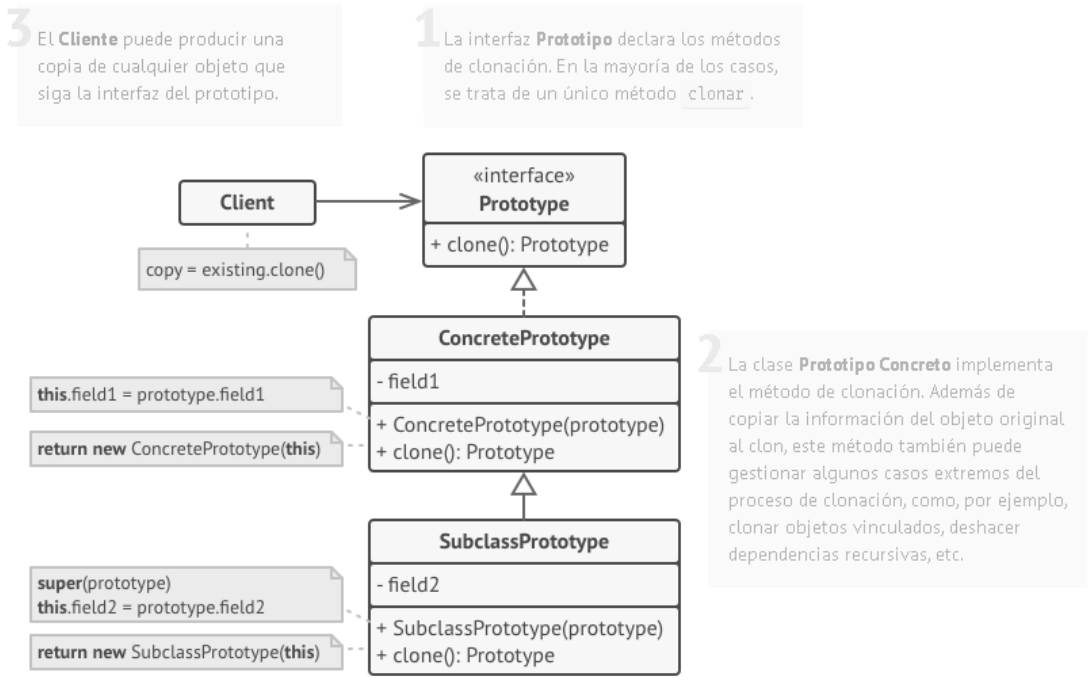
1.4) Prototype: permite copiar objetos existentes sin que el código dependa de sus clases.

En lugar de crear un objeto desde cero con la palabra clave new, este patrón delega el proceso de creación al propio objeto que se quiere duplicar, pidiéndole que devuelva su propia copia exacta.

Sirve en caso de querer crear muchos objetos con leves diferencias entre ellos.

Ejemplo: duplicar enemigos en un videojuego copiando un prototipo base con sus atributos actuales.

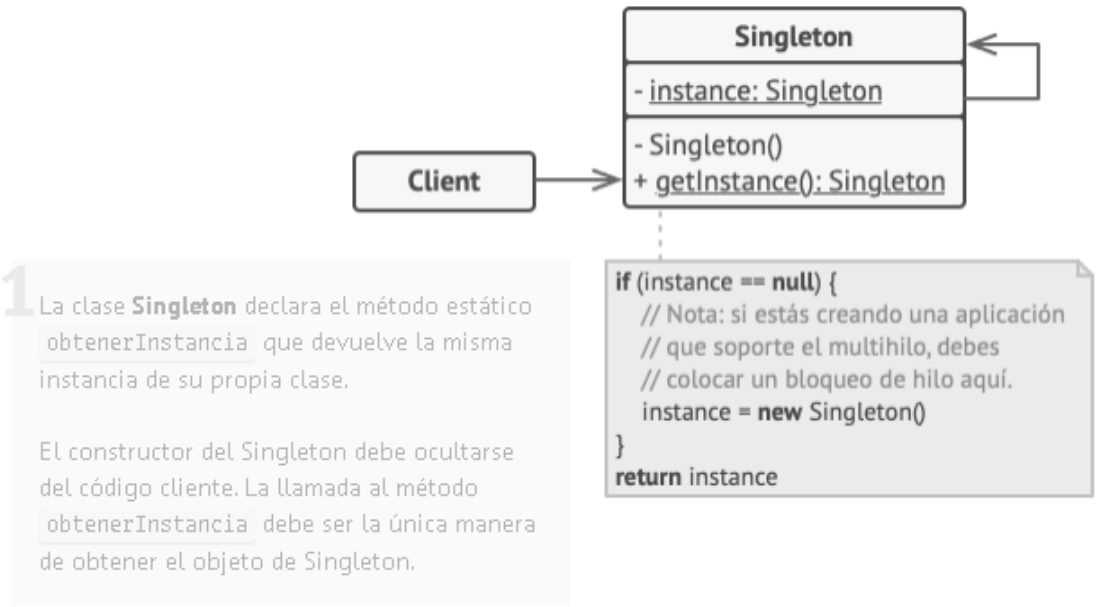
Diagrama:



1.5) Sigleton: patrón de diseño creacional que nos permite asegurarnos de que una clase tenga una única instancia, a la vez que proporciona un punto de acceso global a dicha instancia.
Es ideal utilizarlo en contextos donde un recurso compartido de manera centralizada necesita ser controlado estrictamente para evitar inconsistencias o consumo innecesario de recursos.

Ejemplos: gestores de configuración, conexiones a BD, sistemas de registro (*Loggers*), gestores de caché.

Diagrama:



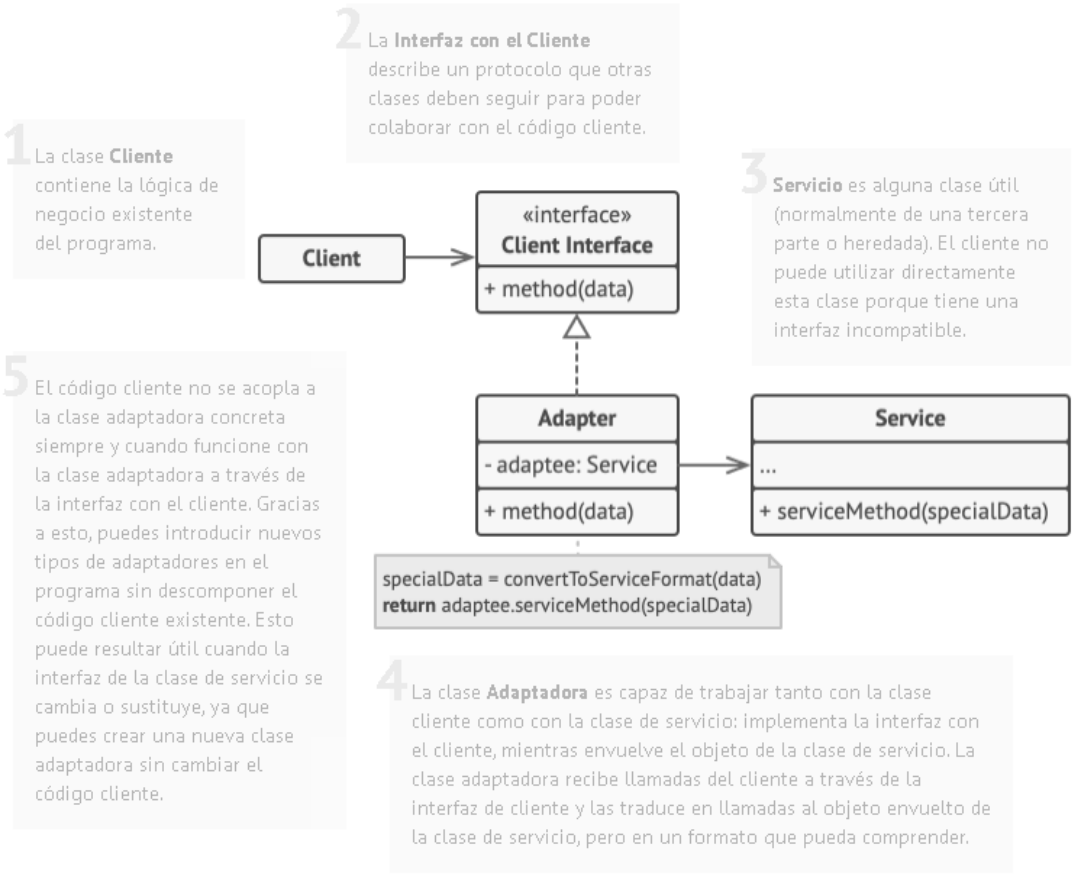
2. **Estructurales:** se enfocan en cómo se combinan clases y objetos para formar estructuras más grandes y complejas, garantizando que, si una parte cambia, toda la estructura no se vea afectada.

2.1) Adapter: permite que objetos con interfaces incompatibles colaboren entre sí. Un adaptador envuelve a uno de los objetos y traduce las llamadas de la interfaz que espera el cliente a la interfaz del objeto adaptado.

Cuándo conviene usarlo: cuando se quiere reutilizar una clase existente (heredada o de terceros) cuya interfaz no coincide con la que necesita el resto del código y no es posible o conveniente modificarla.

Ejemplo: adaptar una pasarela de pago antigua para que funcione con la nueva arquitectura de la aplicación.

Diagrama:

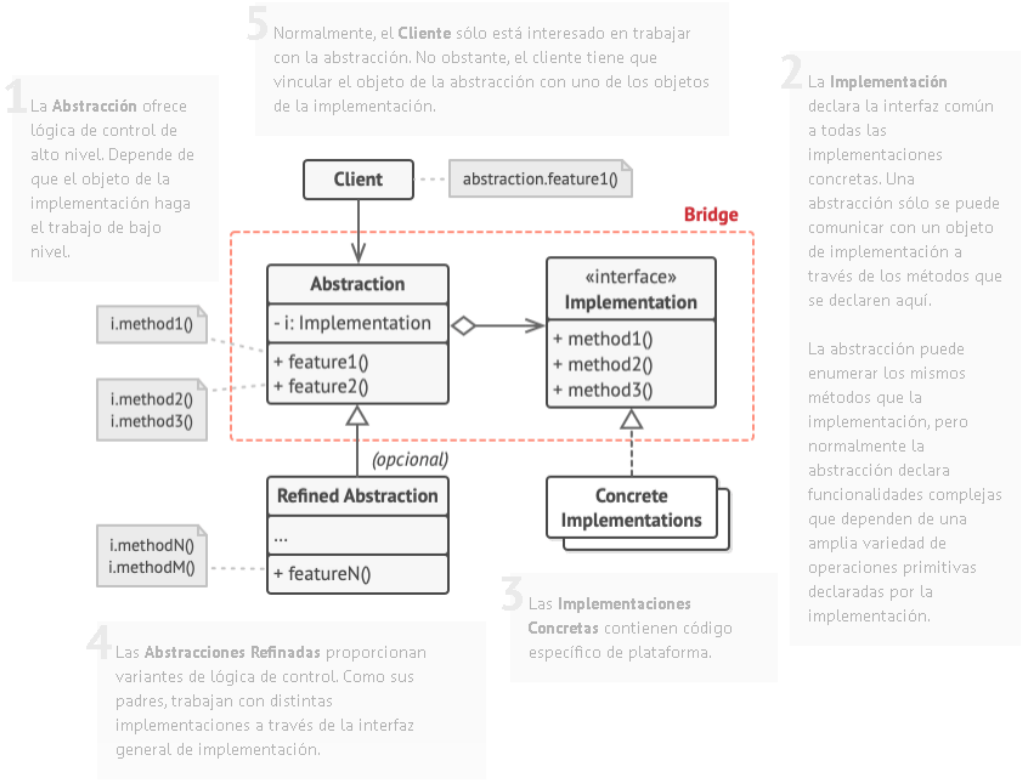


2.2) Bridge: divide una clase grande, o un conjunto de clases estrechamente relacionadas, en dos jerarquías independientes (abstracción e implementación) que pueden desarrollarse por separado. La abstracción delega el trabajo real en un objeto de implementación.

Cuándo conviene usarlo: cuando una clase varía en dos dimensiones independientes y combinarlas por herencia generaría una explosión de subclases, o cuando se necesita cambiar la implementación en tiempo de ejecución (por ejemplo, aplicaciones multiplataforma).

Ejemplo: controles remotos (abstracción) y dispositivos electrónicos (implementación).

Diagrama:

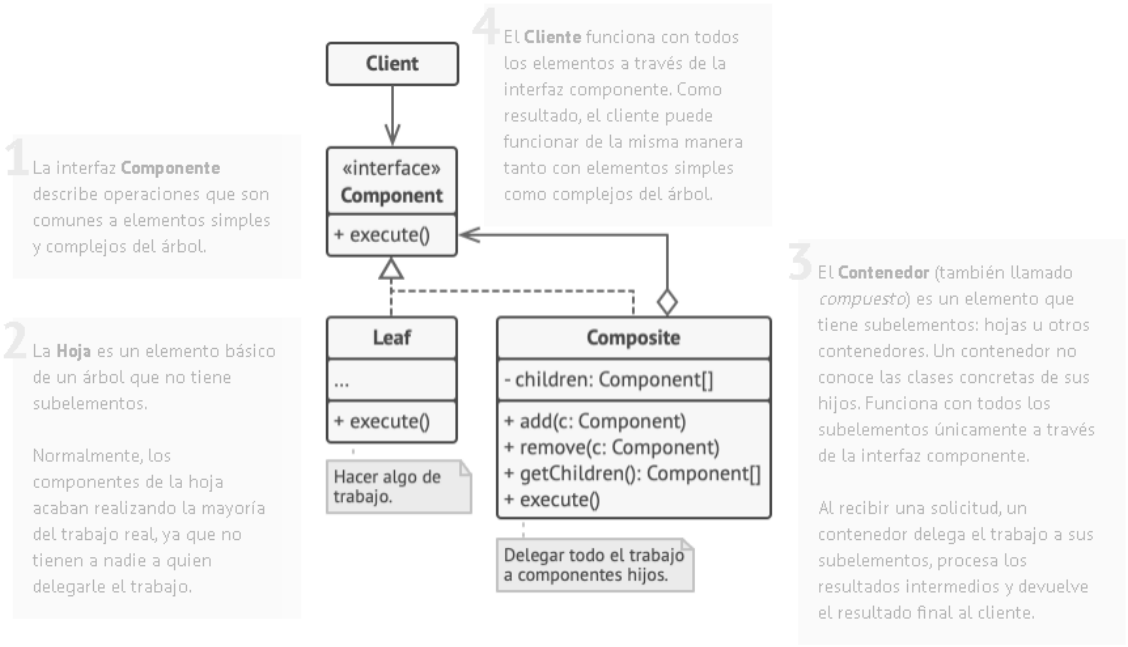


2.3) Composite: permite componer objetos en estructuras de árbol y trabajar con ellas como si fueran objetos individuales. Hojas y compuestos comparten una interfaz común, por lo que el cliente los trata de manera uniforme.

Cuándo conviene usarlo: cuando el modelo es naturalmente jerárquico (relación parte-todo) y el cliente debería poder tratar de la misma forma a un elemento simple y a un grupo de elementos.

Ejemplo: un sistema de archivos con carpetas que contienen archivos u otras carpetas.

Diagrama:

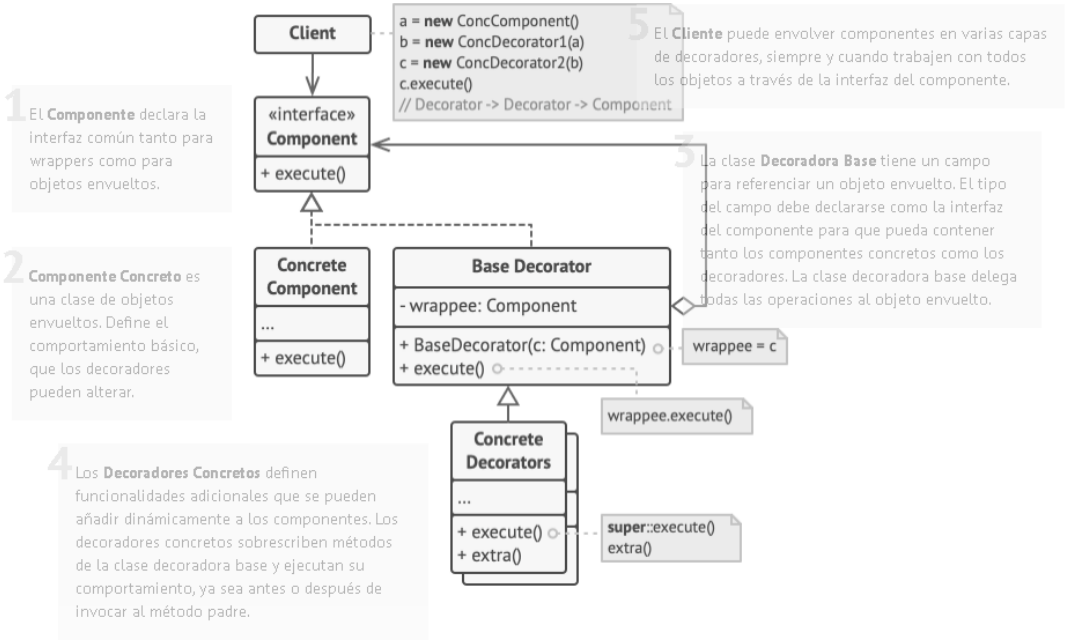


2.4) Decorator: permite añadir comportamientos a un objeto colocándolo dentro de envoltorios que implementan su misma interfaz, sin modificar su clase ni recurrir a la herencia.

Cuándo conviene usarlo: cuando hay que agregar responsabilidades a objetos de forma dinámica y combinable, o cuando extender por herencia es impracticable o produciría demasiadas subclases.

Ejemplo: agregar complementos (leche, jarabe, crema) a una taza de café básica.

Diagrama:

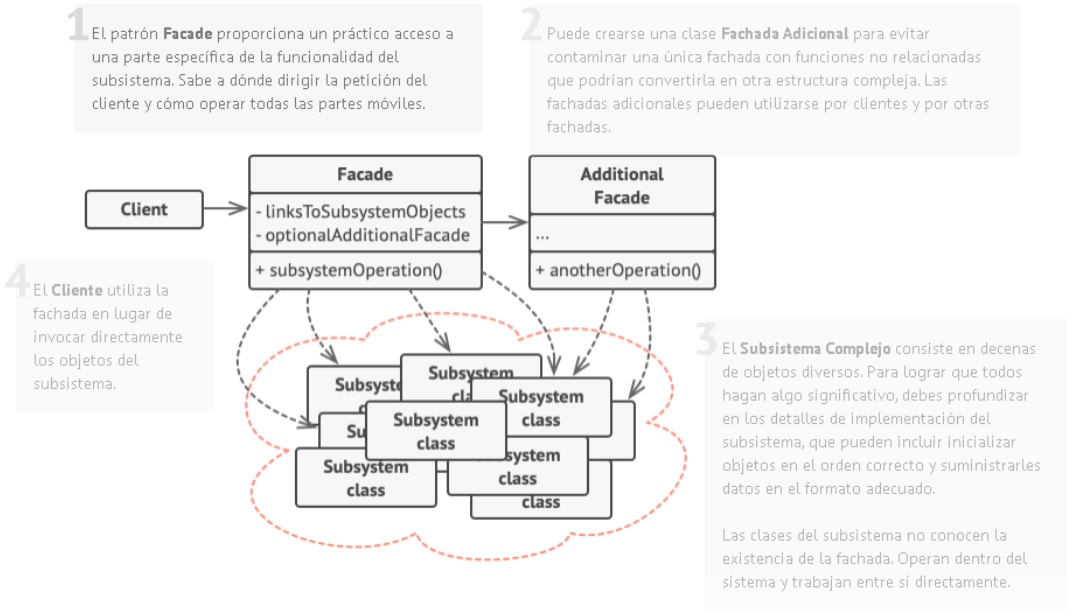


2.5) Facade: proporciona una interfaz simplificada a una biblioteca, un framework o cualquier conjunto complejo de clases (subsistema), exponiendo solo lo que el cliente realmente necesita.

Cuándo conviene usarlo: cuando se necesita una interfaz limitada pero sencilla para un subsistema complejo, o para reducir el acoplamiento entre el cliente y las clases internas, organizando el subsistema en capas.

Ejemplo: un botón de "Encender cine en casa" que coordina la TV, el amplificador y el reproductor.

Diagrama:

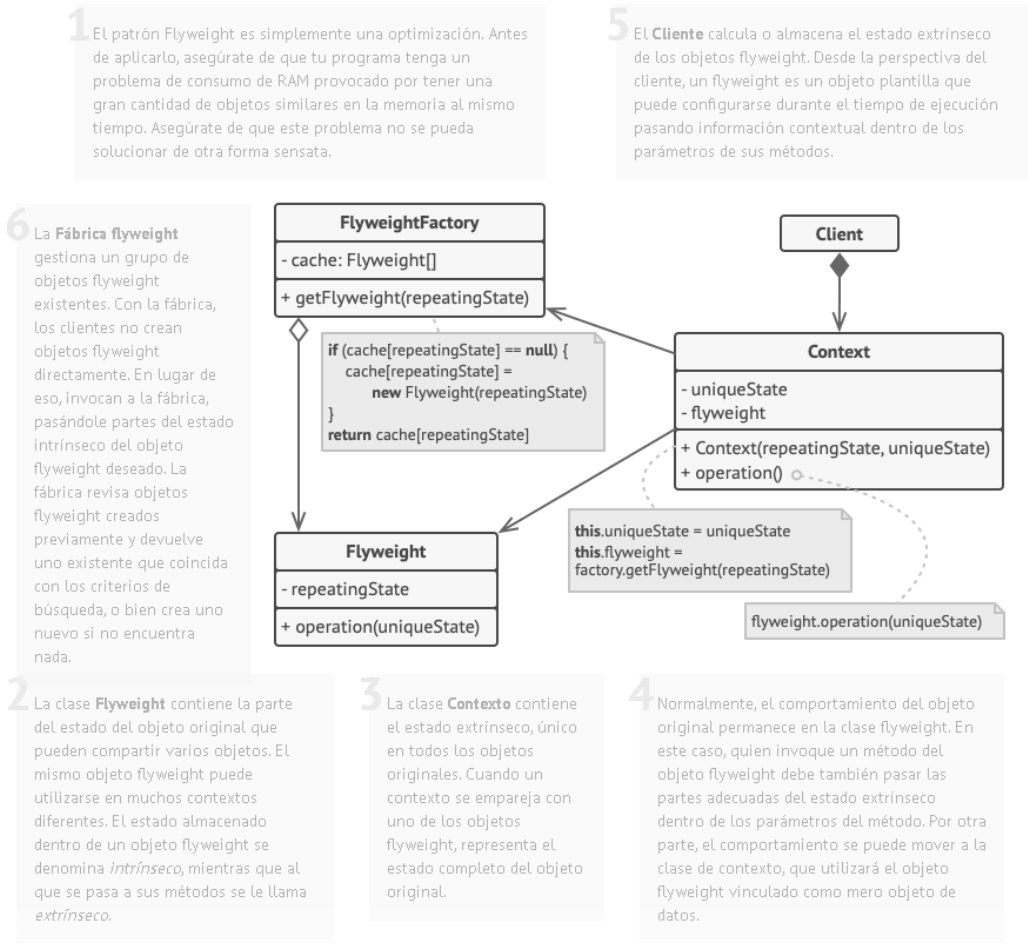


2.6) Flyweight: permite mantener más objetos en memoria compartiendo las partes comunes del estado (estado intrínseco) entre muchos objetos, en lugar de almacenar todos los datos en cada uno. El estado que varía (extrínseco) se pasa como parámetro.

Cuándo conviene usarlo: cuando la aplicación debe crear una enorme cantidad de objetos similares que agotarían la memoria disponible y buena parte de su estado puede separarse y compartirse.

Ejemplo: los caracteres en un procesador de texto (se comparte la fuente, cambiando solo la posición).

Diagrama:



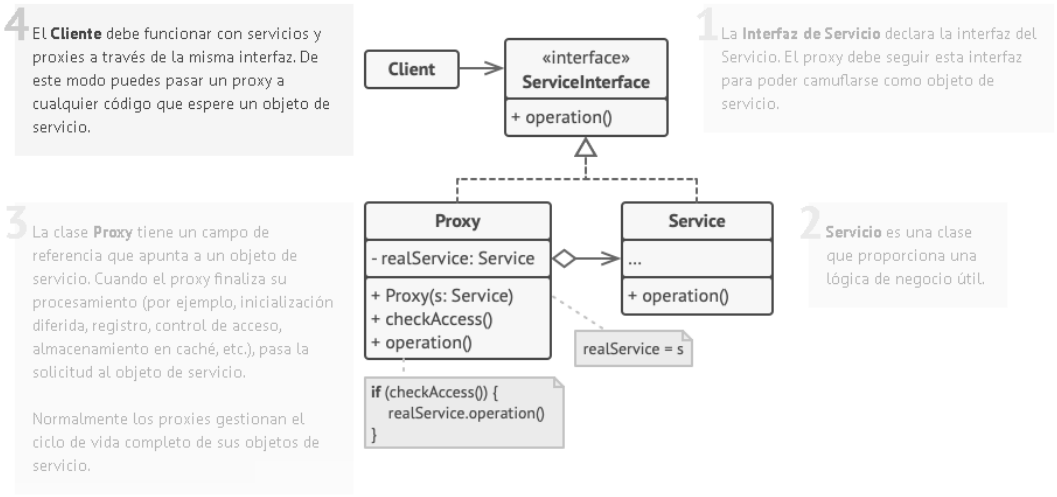
2.7) Proxy: proporciona un sustituto o intermediario de otro objeto que controla el acceso al original, permitiendo ejecutar acciones antes o después de que la solicitud llegue a él.

Cuándo conviene usarlo: cuando se necesita inicialización diferida (proxy virtual), control de acceso o permisos (proxy de protección), acceso a un objeto remoto, caché de resultados o registro de llamadas, sin cambiar el objeto original.

Ejemplo: una imagen virtual que solo descarga el archivo pesado cuando se visualiza en pantalla (*Lazy Loading)*

3. **De comportamiento:** se ocupan de la asignación eficiente de responsabilidades y la comunicación entre objetos, facilitando la interacción flexible y desacoplada.

Diagrama:

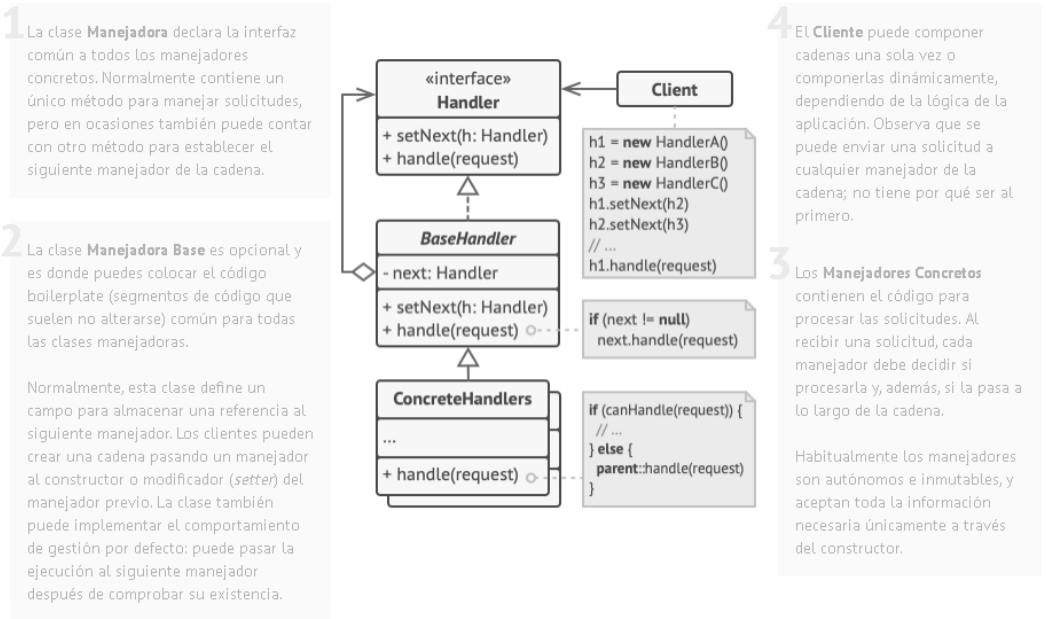


3.1) Chain of Responsibility: permite pasar solicitudes a lo largo de una cadena de manejadores. Cada manejador decide si procesa la solicitud o si la pasa al siguiente de la cadena.

Cuándo conviene usarlo: cuando más de un objeto puede atender una solicitud y el manejador adecuado no se conoce de antemano, o cuando el conjunto y el orden de los manejadores deben poder cambiar dinámicamente.

Ejemplo: un sistema de soporte técnico con niveles (operador, técnico, especialista) donde cada nivel resuelve el problema o lo escala al siguiente; o los filtros de una petición web (autenticación, autorización, validación).

Diagrama:

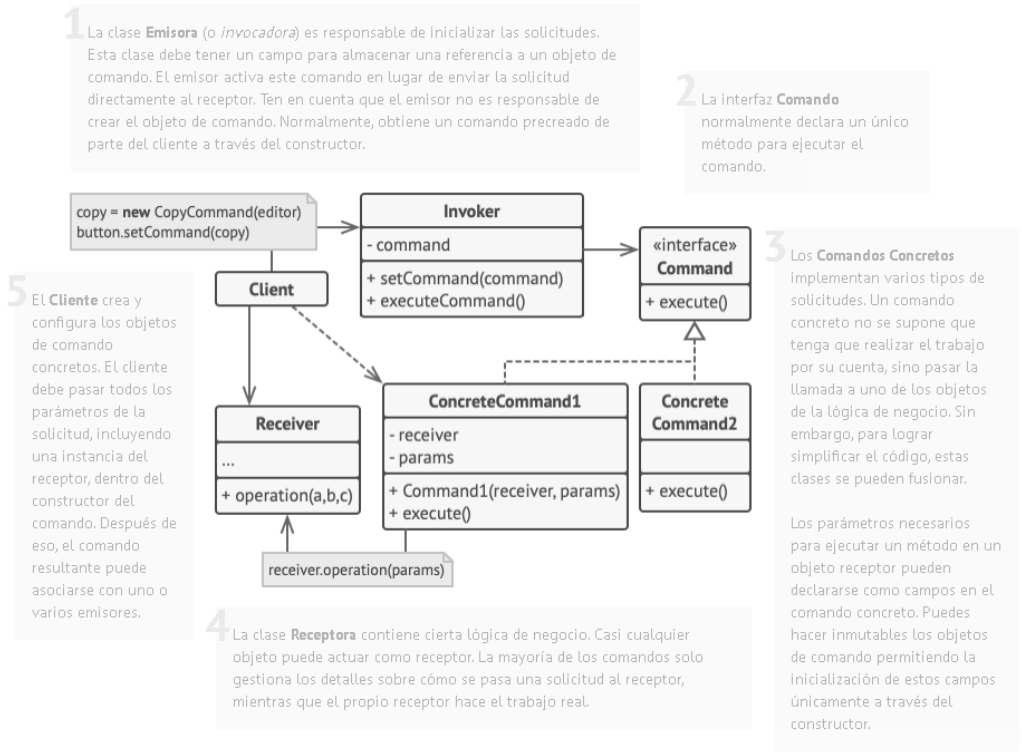


3.2) Command: convierte una solicitud en un objeto independiente que contiene toda la información necesaria para ejecutarla. Esto permite parametrizar métodos con acciones, encolarlas, registrarlas y soportar operaciones de deshacer.

Cuándo conviene usarlo: cuando se quiere desacoplar al objeto que invoca una operación del que la ejecuta, o implementar deshacer/rehacer, colas de tareas, ejecución diferida o historial de operaciones.

Ejemplo: los botones y atajos de un editor de texto (copiar, pegar, deshacer), que ejecutan comandos sin conocer cómo se realizan; o la comanda que el mozo entrega a la cocina.

Diagrama:

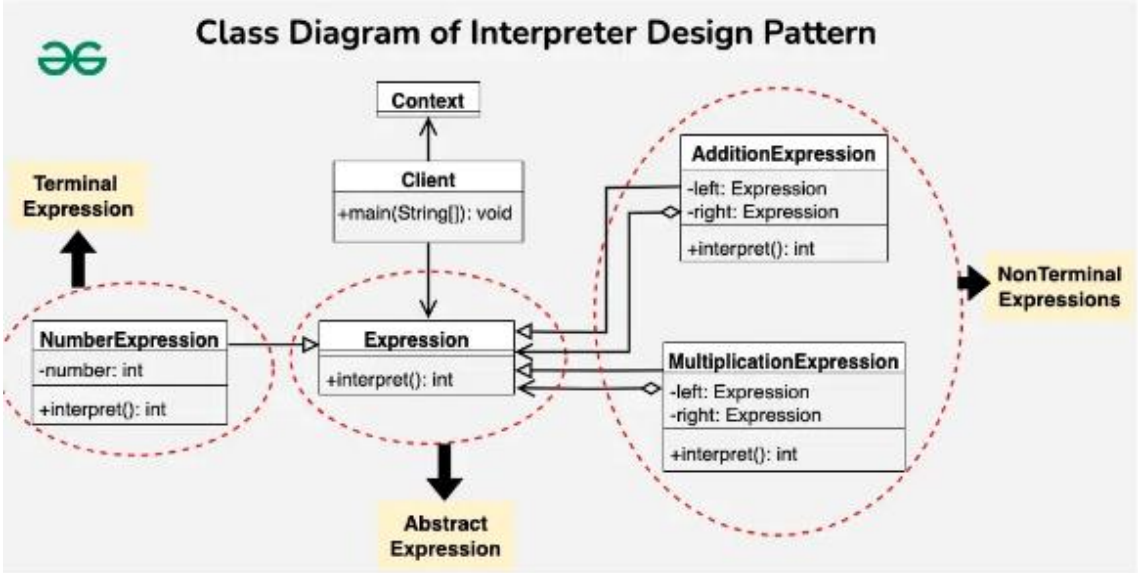


3.3) Interpreter: dado un lenguaje, define una representación de su gramática junto con un intérprete que usa esa representación para interpretar sentencias del lenguaje.

Cuándo conviene usarlo: cuando hay que evaluar con frecuencia sentencias de un lenguaje simple y bien definido, cuya gramática pueda representarse como un árbol de expresiones; no es recomendable para gramáticas complejas ni cuando el rendimiento es crítico.

Ejemplo: una calculadora que evalúa expresiones como 3 + 4 * 2, donde los números son expresiones terminales y las operaciones son expresiones no terminales; o el motor de expresiones regulares.

Diagrama:

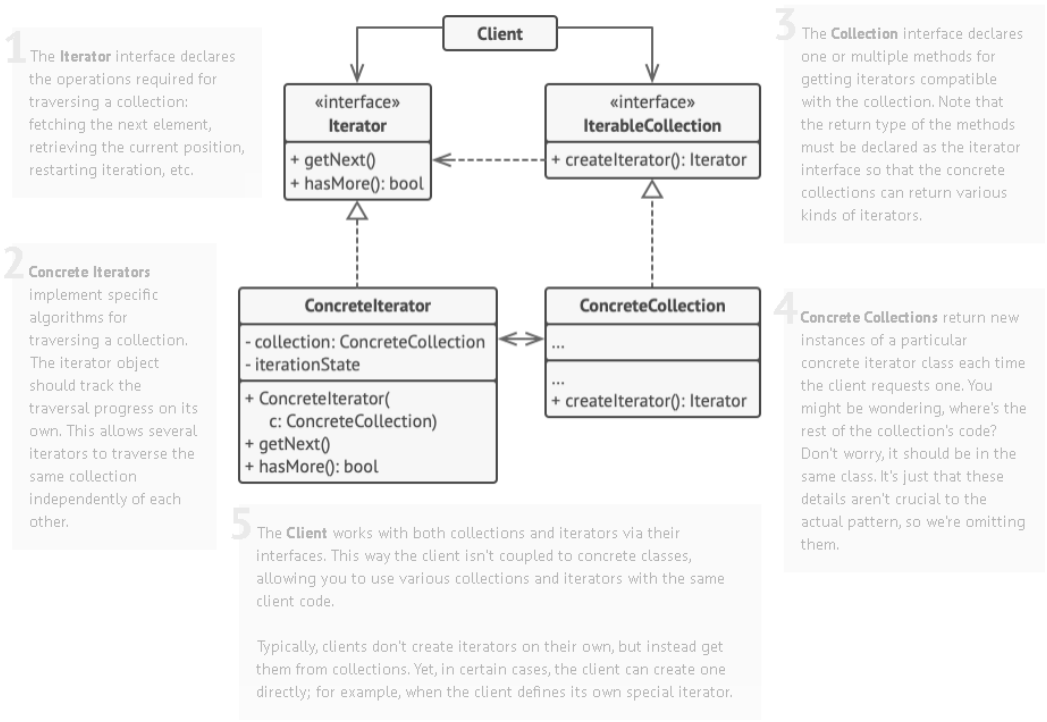


3.4) Iterator: permite recorrer los elementos de una colección sin exponer su representación subyacente (lista, pila, árbol, etc.).

Cuándo conviene usarlo: cuando se necesita acceder a los elementos de una colección de forma uniforme, ocultar su estructura interna al cliente o admitir varios recorridos simultáneos sobre la misma colección.

Ejemplo: recorrer una lista de reproducción o los resultados de una búsqueda; en Java, la interfaz Iterator de las colecciones y el bucle for-each.

Diagrama:

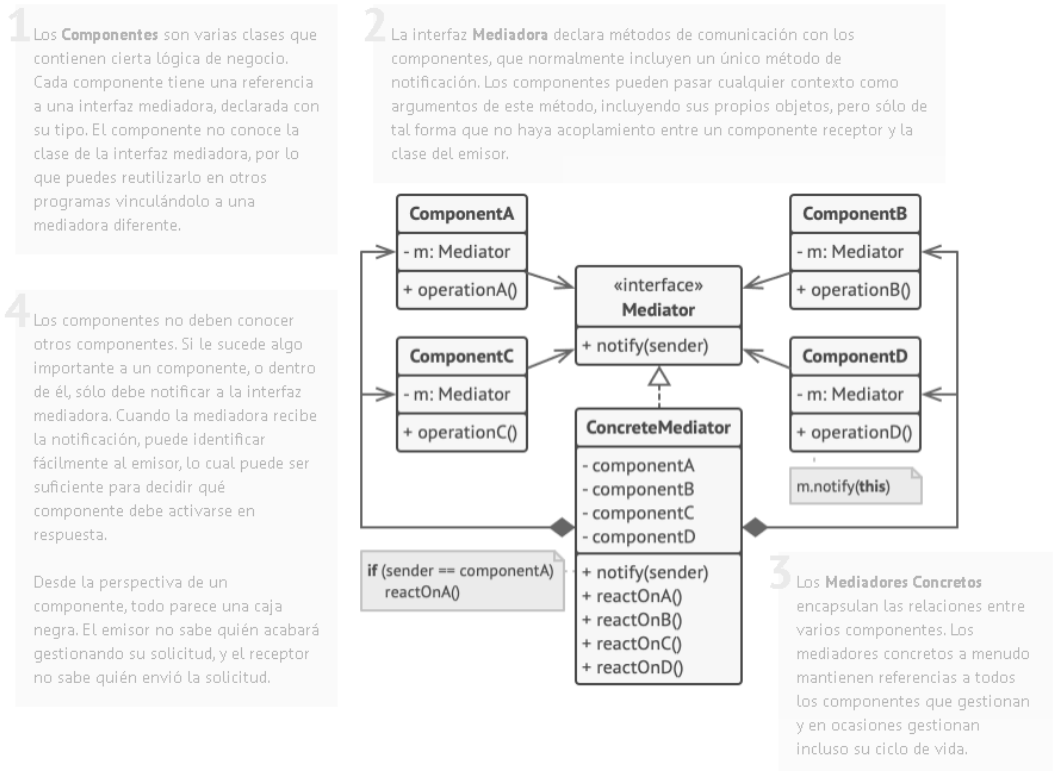


3.5) Mediator: reduce las dependencias caóticas entre objetos restringiendo la comunicación directa y obligándolos a colaborar únicamente a través de un objeto mediador.

Cuándo conviene usarlo: cuando muchas clases están fuertemente acopladas entre sí, lo que dificulta modificarlas o reutilizarlas, o cuando la lógica de interacción entre componentes conviene centralizarla en un solo lugar.

Ejemplo: una torre de control de aeropuerto que coordina a los aviones, que no se comunican directamente entre sí; o un formulario donde el diálogo coordina campos, casillas y botones.

Diagrama:

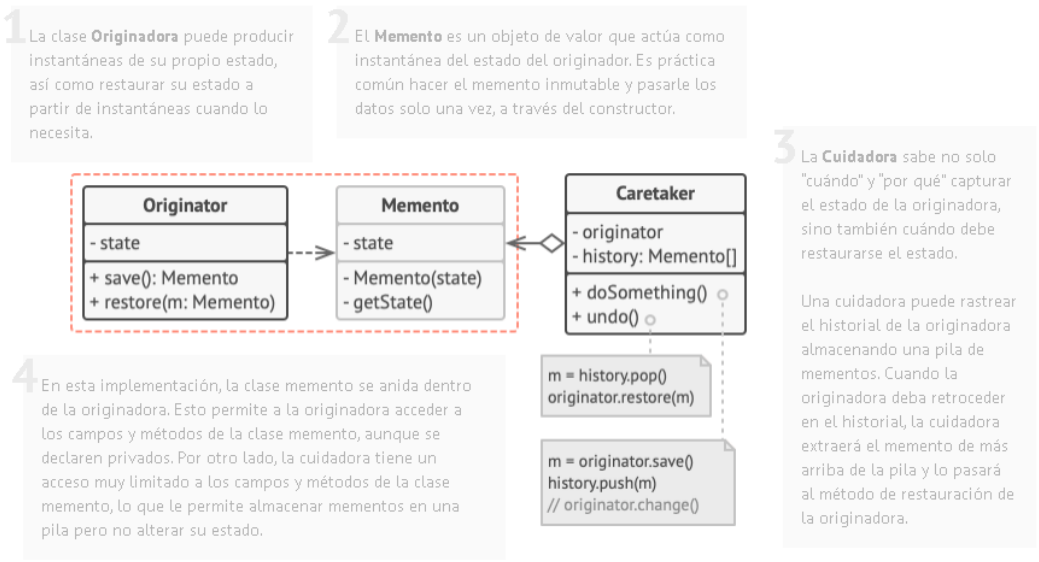


3.6) Memento: permite guardar y restaurar el estado previo de un objeto sin revelar los detalles de su implementación, preservando el encapsulamiento.

Cuándo conviene usarlo: cuando se necesita tomar instantáneas del estado de un objeto para poder restaurarlo más tarde (deshacer, puntos de control) y acceder directamente a ese estado violaría su encapsulamiento.

Ejemplo: la función deshacer de un editor de texto, o los puntos de guardado de un videojuego.

Diagrama:

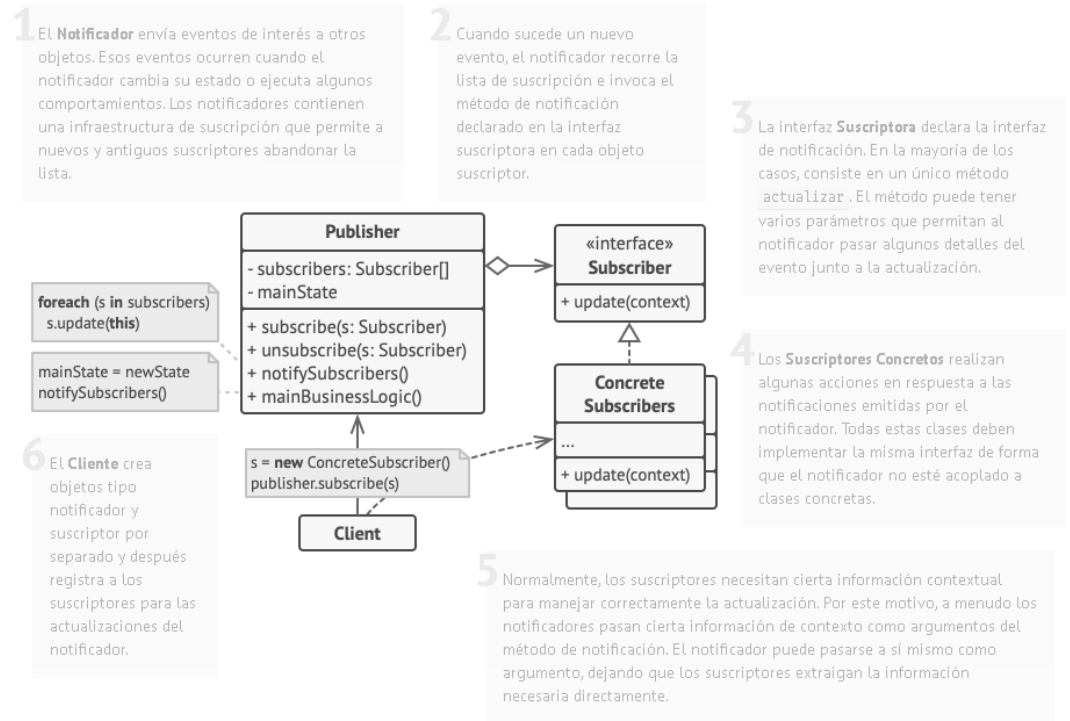


3.7) Observer: define un mecanismo de suscripción para notificar a varios objetos (observadores) sobre cualquier evento que le ocurra al objeto que están observando (sujeto).

Cuándo conviene usarlo: cuando el cambio de estado de un objeto debe reflejarse en otros cuya cantidad es desconocida o cambia dinámicamente, manteniéndolos desacoplados del sujeto.

Ejemplo: la suscripción a un canal o a un boletín: cada vez que se publica contenido nuevo se notifica a todos los suscriptores; también el modelo de eventos de las interfaces gráficas y el patrón MVC.

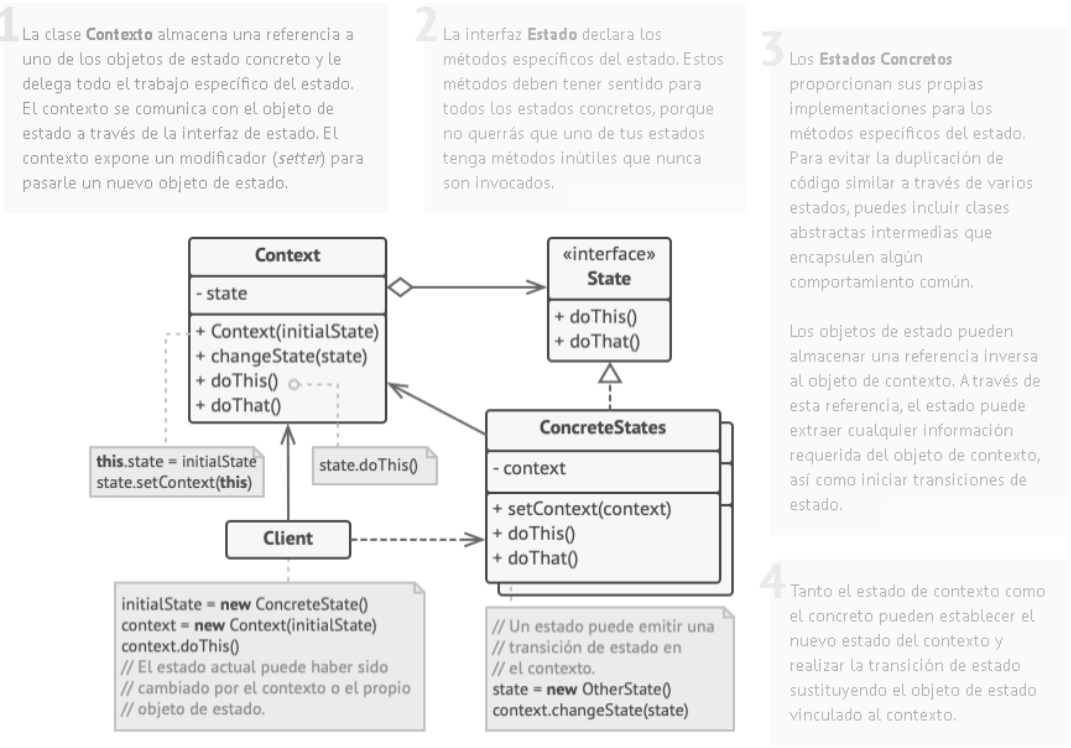
Diagrama:



3.8) State: permite que un objeto altere su comportamiento cuando cambia su estado interno, de modo que parezca que cambió de clase. Cada estado se encapsula en su propia clase.

Cuándo conviene usarlo: cuando el comportamiento de un objeto depende de su estado y hay que cambiar entre estados en tiempo de ejecución, o cuando el código se llena de condicionales extensos que verifican el estado actual.

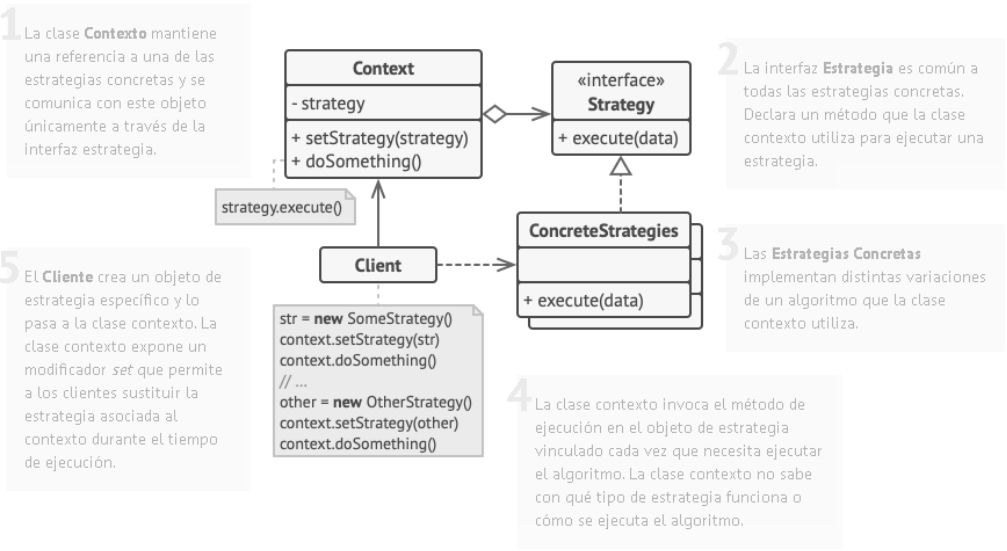
Ejemplo: un reproductor de música cuyos botones se comportan distinto según esté reproduciendo, en pausa o detenido; o un pedido que pasa por los estados pendiente, pagado y enviado.



3.9) Strategy: define una familia de algoritmos, coloca cada uno en una clase separada y hace que sus objetos sean intercambiables.

Cuándo conviene usarlo: cuando existen varias variantes de un mismo algoritmo y se quiere elegir una en tiempo de ejecución, o para eliminar condicionales extensos que seleccionan entre distintos comportamientos.

Ejemplo: una aplicación de mapas que calcula la ruta según la estrategia elegida (a pie, en auto o en transporte público); o distintos métodos de pago en una tienda en línea.

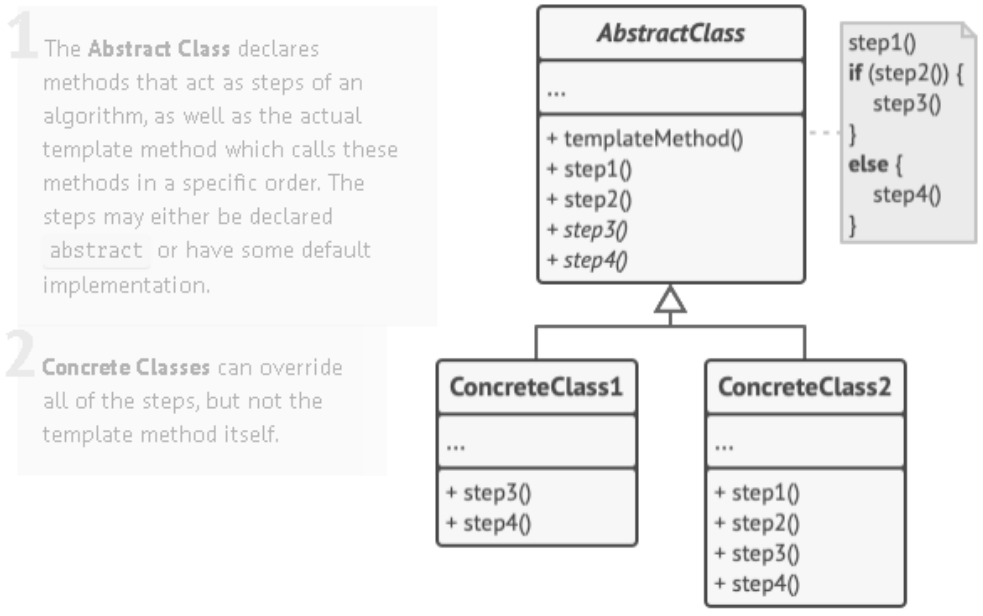


3.10) Template Method: define el esqueleto de un algoritmo en una superclase, pero permite que las subclases sobrescriban pasos específicos sin cambiar la estructura general del algoritmo.

Cuándo conviene usarlo: cuando varias clases tienen algoritmos casi idénticos que difieren solo en algunos pasos, o cuando se quiere permitir a los clientes extender únicamente ciertos pasos de un algoritmo.

Ejemplo: un procesador de documentos que abre el archivo, extrae los datos, los analiza y los cierra, donde cada formato (CSV, PDF, DOC) implementa solo la extracción; o la preparación de bebidas calientes (té y café).

Diagrama:



3.11) Visitor: permite separar los algoritmos de los objetos sobre los que operan, agregando nuevas operaciones a una jerarquía de clases sin modificarlas.

Cuándo conviene usarlo: cuando se debe realizar una operación sobre todos los elementos de una estructura compleja formada por clases distintas, y la jerarquía es estable pero las operaciones cambian con frecuencia.

Ejemplo: exportar a XML o JSON los distintos elementos de un árbol de figuras geométricas (círculo, rectángulo, polígono) sin agregar un método de exportación a cada clase.

Diagrama:

